

Struktur Khusus & Reuse

Menyimpan jawaban, menggabungkan kelompok, dan menyimpan kata berdasarkan prefix.

Tujuan dokumen ini: bukan menghafal tanda-tanda soal, tapi memahami gerak berpikir pattern. Setiap pattern dijelaskan dengan contoh dunia nyata, contoh teknologi, konsep komputasi, alasan kenapa contoh itu sama dengan pattern, dan template Dart minimal.

Cara Membaca Dokumen Ini

Jangan baca seperti kamus keyword. Baca seperti latihan membangun intuisi. Pattern LeetCode itu mirip alat kerja: tiap alat punya cara bergerak, situasi natural, dan bentuk data yang cocok.

Untuk tiap pattern, pakai urutan baca ini: makna manusiawi -> contoh dunia nyata -> contoh teknologi -> konsep komputasi -> bentuk LeetCode -> template Dart -> pertanyaan cek pemahaman.

Pattern	Makna inti
DP	menyimpan jawaban subproblem agar tidak dihitung ulang
Union Find	menggabungkan dan mengecek kelompok
Trie	menyimpan kata berdasarkan jalur prefix

DP - menyimpan jawaban subproblem agar tidak dihitung ulang

Makna manusiawinya

Dynamic Programming adalah cara menghindari mikir ulang subproblem yang sama. Kalau masalah besar bisa dipecah menjadi masalah kecil yang berulang, simpan jawabannya.

Contoh nyata non-teknologi

Menghitung biaya komponen project

Beberapa fitur memakai modul yang sama. Kalau biaya modul sudah dihitung, jangan hitung ulang setiap kali fitur lain membutuhkannya.

Rute perjalanan dengan sub-rute sama

Kalau beberapa rute melewati kota yang sama, informasi biaya dari kota itu ke tujuan bisa disimpan.

Contoh nyata di dunia teknologi

Memoization dalam app

Hasil expensive computation disimpan agar render berikutnya lebih cepat.

Build system

File yang tidak berubah tidak dibuild ulang; hasil sebelumnya dipakai lagi.

Konsep komputasinya

Secara komputasi, DP butuh state, transition, dan base case. State menjelaskan subproblem. Transition menjelaskan cara membangun jawaban dari state lain. Base case adalah jawaban paling kecil.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena ada pengulangan subproblem. DP bukan magic; DP adalah cache berpola untuk keputusan/hasil yang saling bergantung.

Bentuknya di LeetCode

- Climbing Stairs
- House Robber
- Coin Change
- Longest Common Subsequence
- Edit Distance
- Stock DP

Template Dart minimal

```
int climbStairs(int n) {  
  if (n <= 2) return n;  
  int prev2 = 1;  
  int prev1 = 2;  
  
  for (int i = 3; i <= n; i++) {  
    final current = prev1 + prev2;
```

```
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Kalau masih bingung, tanya begini

- Apa subproblem yang berulang?
- State-nya apa?
- Transition-nya apa?
- Base case-nya apa?
- Apakah perlu top-down atau bottom-up?

Mini drill pemahaman

1. Untuk Climbing Stairs, state dan transition-nya apa?
2. Kenapa Fibonacci naive lambat?
3. Apa beda DP dan greedy?

Union Find - menggabungkan dan mengecek kelompok

Makna manusiawinya

Union Find atau DSU mengelola kelompok dinamis. Pertanyaan utamanya: dua item ini satu kelompok tidak? Kalau belum, gabungkan.

Contoh nyata non-teknologi

Menggabungkan grup WhatsApp yang overlap

Kalau A satu grup dengan B, dan B satu grup dengan C, maka A dan C terhubung dalam komunitas yang sama.

Kartu keluarga / komunitas

Orang bisa digabung ke satu keluarga/komunitas. Kita ingin cepat tahu dua orang berada di kelompok yang sama atau tidak.

Contoh nyata di dunia teknologi

Accounts Merge

Email-email yang overlap berarti akun-akun itu milik orang yang sama.

Network connectivity

Cek apakah dua server berada di komponen jaringan yang sama setelah beberapa koneksi ditambahkan.

Konsep komputasinya

Secara komputasi, DSU punya find untuk mencari root kelompok dan union untuk menggabungkan dua kelompok. Optimasi path compression membuat find sangat cepat.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena identitas kelompok lebih penting daripada jalur detail. Kita tidak selalu butuh path dari A ke B, hanya apakah A dan B connected.

Bentuknya di LeetCode

- Number of Provinces
- Redundant Connection
- Accounts Merge
- Graph Valid Tree
- Connected Components

Template Dart minimal

```
class DSU {  
  late List<int> parent;  
  late List<int> rank;  
  
  DSU(int n) {  
    parent = List.generate(n, (i) => i);  
    rank = List.filled(n, 0);  
  }  
  
  int find(int x) {
```

```

        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    bool union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank[ra] < rank[rb]) {
            parent[ra] = rb;
        } else if (rank[ra] > rank[rb]) {
            parent[rb] = ra;
        } else {
            parent[rb] = ra;
            rank[ra]++;
        }
        return true;
    }
}

```

Kalau masih bingung, tanya begini

- Apakah fokusnya kelompok/komponen?
- Apakah banyak operasi merge dan cek same group?
- Apakah path detail tidak penting?

Mini drill pemahaman

1. Bedakan DFS connected component dan DSU.
2. Kenapa path compression membantu?
3. Kapan union mengembalikan false berguna?

Trie - menyimpan kata berdasarkan jalur prefix

Makna manusiawinya

Trie menyimpan string sebagai jalur karakter. Kata-kata dengan prefix sama berbagi jalur yang sama. Ini cocok untuk prefix search dan dictionary.

Contoh nyata non-teknologi

Rak kamus berdasarkan huruf depan

Semua kata berawalan ca masuk cabang c -> a. Cat, car, care berbagi jalur awal.

Autocomplete pikiran

Saat lu mengetik pro, sistem mempersempit kandidat ke kata yang punya prefix pro.

Contoh nyata di dunia teknologi

Search suggestion

E-commerce menampilkan suggestion saat user mengetik huruf demi huruf.

Word Search II

Trie membantu mengecek banyak kata sekaligus saat menyusuri grid huruf.

Konsep komputasinya

Secara komputasi, setiap node punya children dan penanda isWord. Search prefix berjalan $O(\text{panjang prefix})$, bukan $O(\text{jumlah semua kata})$.

Kenapa contoh itu sama dengan pattern ini?

Contohnya sama karena pencarian dibangun dari prefix bertahap. Trie tidak menyimpan kata sebagai daftar flat, tapi sebagai pohon jalur karakter.

Bentuknya di LeetCode

- Implement Trie
- Design Add and Search Words
- Search Suggestions System
- Replace Words
- Word Search II

Template Dart minimal

```
class TrieNode {
  final Map<String, TrieNode> children = {};
  bool isWord = false;
}

class Trie {
  final root = TrieNode();

  void insert(String word) {
    var node = root;
    for (final ch in word.split('')) {
      node = node.children.putIfAbsent(ch, () => TrieNode());
    }
  }
}
```

```
        node.isWord = true;
    }

    bool startswith(String prefix) {
        var node = root;
        for (final ch in prefix.split('')) {
            final next = node.children[ch];
            if (next == null) return false;
            node = next;
        }
        return true;
    }
}
```

Kalau masih bingung, tanya begini

- Apakah banyak query prefix?
- Apakah dictionary besar?
- Apakah kata berbagi awalan?
- Apakah search biasa dengan HashSet tidak cukup?

Mini drill pemahaman

1. Kenapa autocomplete cocok dengan Trie?
2. Apa arti isWord?
3. Apa bedanya mencari prefix dan mencari kata penuh?

Penutup

Mastery bukan berarti menghafal semua tanda soal. Mastery berarti saat melihat masalah, lu bisa memodelkan gerak masalahnya: apakah ia menyebar, menyelam, menjaga window, mencari boundary, mengingat key, menggabungkan kelompok, atau menyimpan subproblem.

Cara latihan: ambil satu pattern, pahami analoginya, kerjakan 3-5 soal mudah, tulis ulang konsepnya dengan bahasa sendiri, lalu campur dengan pattern lain supaya otak belajar mengenali bentuk masalah tanpa spoiler.